CSE221 / ALGORITHMS

Brute Force Search with Backtracking

The 15-Puzzle and N-Queens Problems

One blank square creates a branching search



BLANK MOVES: U / D / L / R

THINK FIRST

Which move should we try - and how will the algorithm know when to stop?

- **State:** the exact tile arrangement
- **Choice:** a legal blank move
- **Branch:** every possible continuation

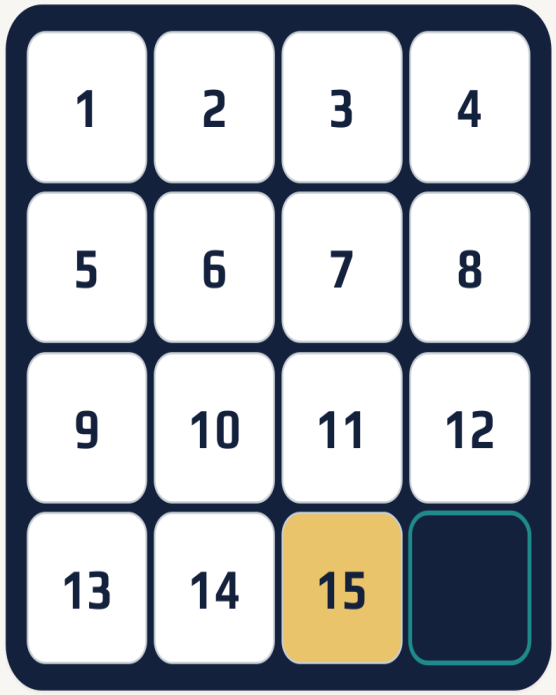
30-second prediction: name one rejection rule.

Model the puzzle before searching it



START STATE

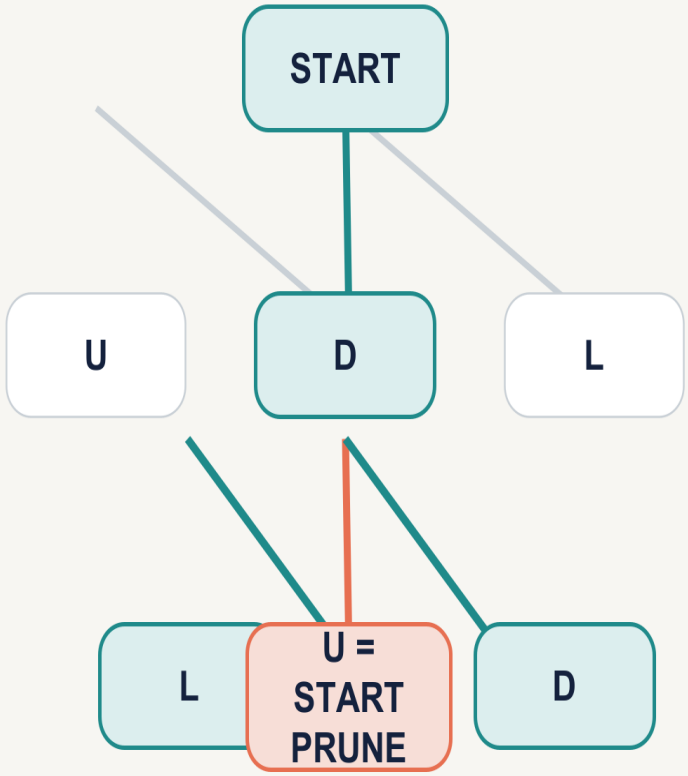
swap blank
with a neighbor



GOAL TEST = TRUE

STATE	16-cell tuple	**ACTIONS**	U / D / L / R	**GOAL**	ordered 1...15 + blank

Backtracking is brute force with rejection rules



repeated state -> prune

Reject a branch when...

- **Illegal:** the blank would leave the board
- **Cyclic:** the state is already on this path
- **Too deep:** the current limit is reached

Undo the move, then try the next choice.

The generator stays brute force; the rejection rules shrink the explored tree.

The recursive loop is choose, check, explore, undo

```
def search(state, depth):
    if state == GOAL: return []

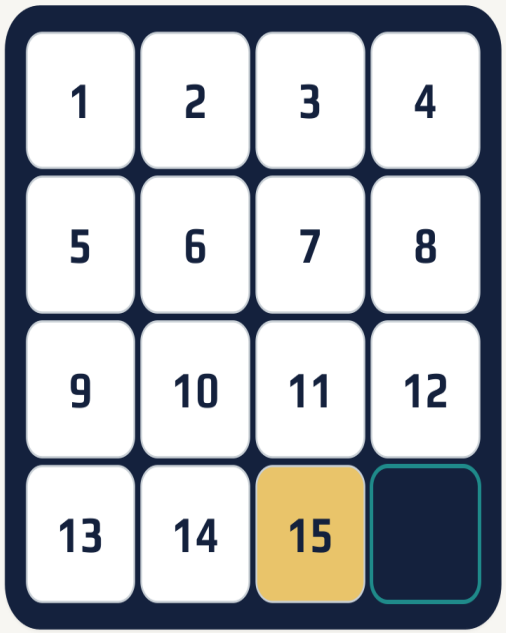
    for move in legal_moves(state):
        next = apply(state, move)       # choose
        if next in on_path: continue    # check

        on_path.add(next)
        path = search(next, depth + 1)  # explore
        on_path.remove(next)            # undo

        if path is not None:
            return [move] + path
    return None
```

1. **CHOOSE**
   Generate a legal move.
2. **CHECK**
   Reject cycles and limits.
3. **EXPLORE**
   Recurse one level deeper.
4. **UNDO**
   Restore path state.

Parity rejects impossible boards before search



SOLVABLE

0 inversions + row 1 = 1



IMPOSSIBLE

1 inversion + row 1 = 2

For a 4 x 4 board

inversions + blank row from the bottom

must be ODD

Convention: goal blank is bottom-right. Count inversions among numbered tiles only.

Check parity once. Save an exponential search.

Backtracking prunes a loop, then continues

ANIMATED SEARCH TRACE

Predict first

After the first D move, why is U rejected?

- Keep the current path in a set.
- Reject any state already on that path.
- Continue with the next legal move.



START

Choose D

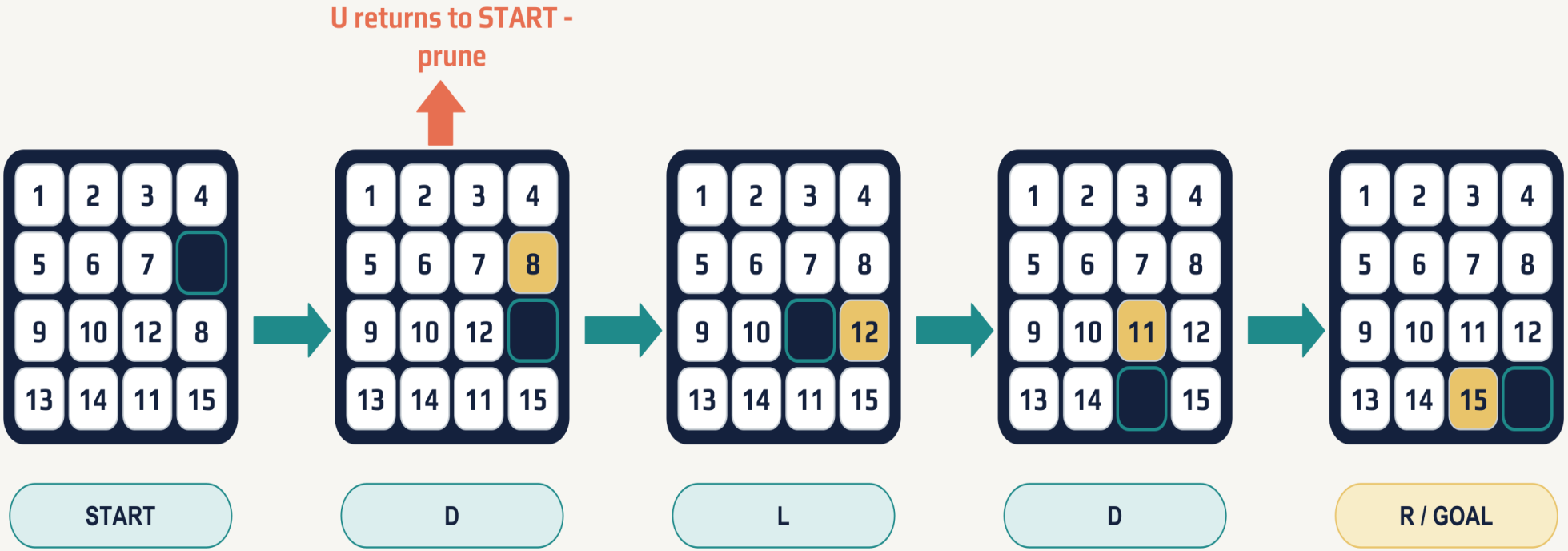
Explore one legal move.

1 / 6

Successful path: D, L, D, R

The animation loops automatically in Slide Show mode.

The recursion stack stores the path to undo



on_path **[START, D, L, D, R]**

Push before recursion. Pop after failure. Keep the moves when the goal returns success.

Depth limits keep DFS from wandering forever

Iterative deepening repeats the search with a larger limit.



Restart on_path for every new limit.

```
for limit in range(MAX_DEPTH
+ 1):
    result = dfs_limited(start,
limit)
    if result is not None:
        return result
```

Why use it?

- DFS-like memory
- complete for finite branching
- shallowest solution for unit costs

Pruning helps, but growth stays exponential

Branching

2 - 4

legal moves from a state
(average 3 by blank position)

Search tree

O(b^d)

depth 4 -> about 3^4 = 81
depth 8 -> about 3^8 = 6,561
depth 12 -> about 3^12 = 531,441

illustrative tree counts before pruning

Reachable states

16! / 2

10,461,394,944,000

Backtracking is practical for shallow examples and teaching. Stronger search is needed for hard instances.

Prune early. Still plan for exponential time.

Safe search tracks path and best depth

```
def dfs(state, depth, limit, on_path, best_depth):
    if state == GOAL: return []
    if depth == limit: return None

    for move in legal_moves(state):
        nxt = apply(state, move)
        if nxt in on_path: continue
        if best_depth.get(nxt, INF) <= depth + 1: continue

        best_depth[nxt] = depth + 1
        on_path.add(nxt)
        suffix = dfs(nxt, depth + 1, limit, on_path, best_depth)
        on_path.remove(nxt)

        if suffix is not None: return [move] + suffix
    return None
```

State key

tuple(board)

on_path

Stops cycles in the current recursive branch.

best_depth

Rejects a state already reached at an equal or shallower depth.

Three bugs can silently break the search

1 State leaks

X

A move mutates the board and is never reversed.

FIX: copy the board or undo the swap.

2 Bad visited rule

X

One global visited set rejects every later revisit.

FIX: use on_path plus best depth.

3 No early guard

X

The search starts without parity or depth checks.

FIX: reject impossible boards; set a limit.

Minimum tests goal | one move | unsolvable parity | immediate reverse cycle

Your turn: choose the shortest first move



BLANK MOVES: U / D / L / R

DEPTH LIMIT = 2

Which first move starts a solution of at most two moves?

Vote: U D L R

- Ignore the move order in code.
- Trace the blank, not the tile.
- Commit before advancing.

Down, then right solves it in two moves



dfs(start, 0) -> D -> dfs(state, 1) -> R -> GOAL

Other first moves may still solve the puzzle - but not within this depth limit.

Use backtracking when rejection rules are clear

A reliable 15-puzzle search

1. **Model** state, moves, and goal
2. **Reject** illegal, cyclic, or too-deep branches
3. **Undo** path membership and any in-place swap
4. **Guard** with parity and a depth strategy
5. **Expect** exponential time in the worst case

EXIT TICKET

What does the on_path set prevent?

Answer in one sentence.

Extension

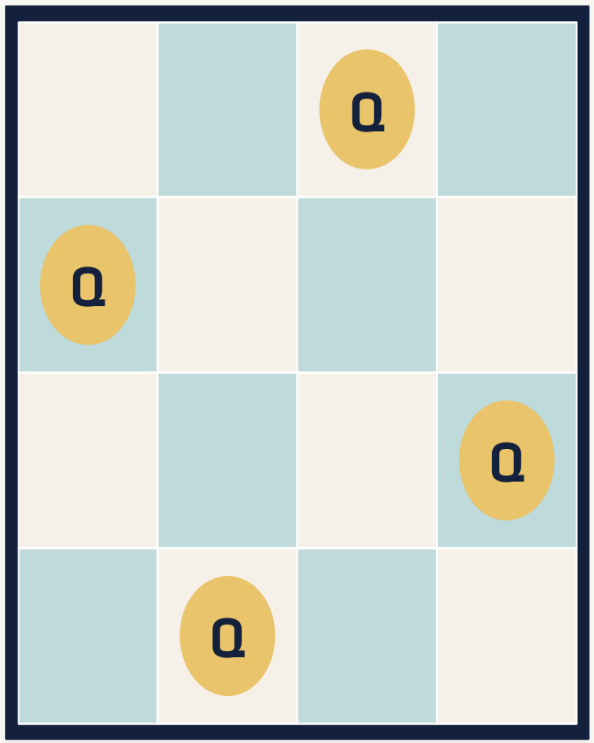
Order moves with a heuristic and compare backtracking with IDA*.

Now switch from moving tiles to placing queens

SECOND FOCUS PROBLEM

The search pattern stays the same.

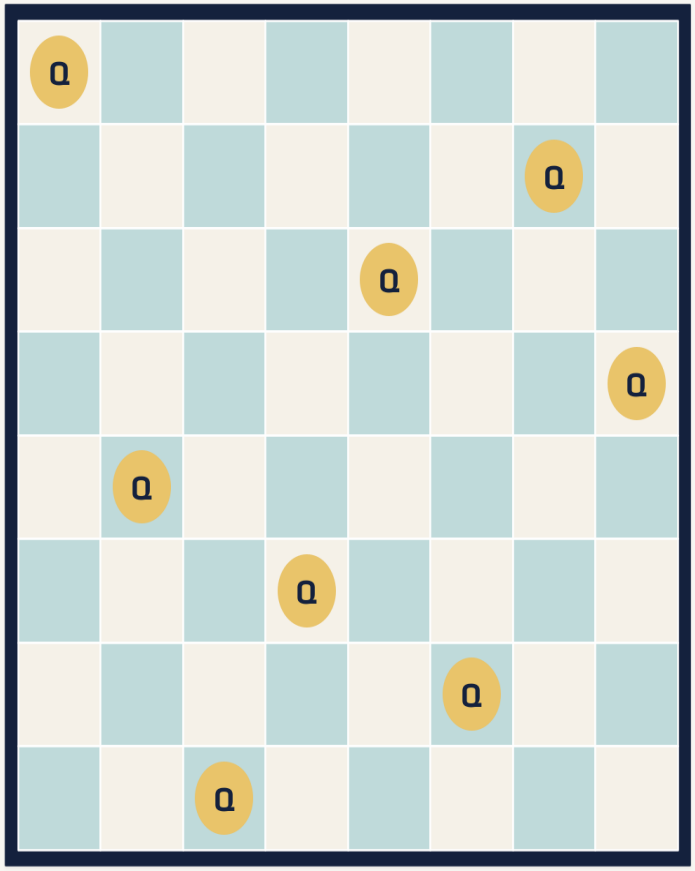
1. Choose a row
2. Check rows and diagonals
3. Explore the next column
4. Undo the queen



4-QUEENS SOLUTION

One queen per column; no shared row or diagonal.

N-Queens turns placement into a constraint search



8-QUEENS EXAMPLE

Goal

Place N queens on an N x N board.

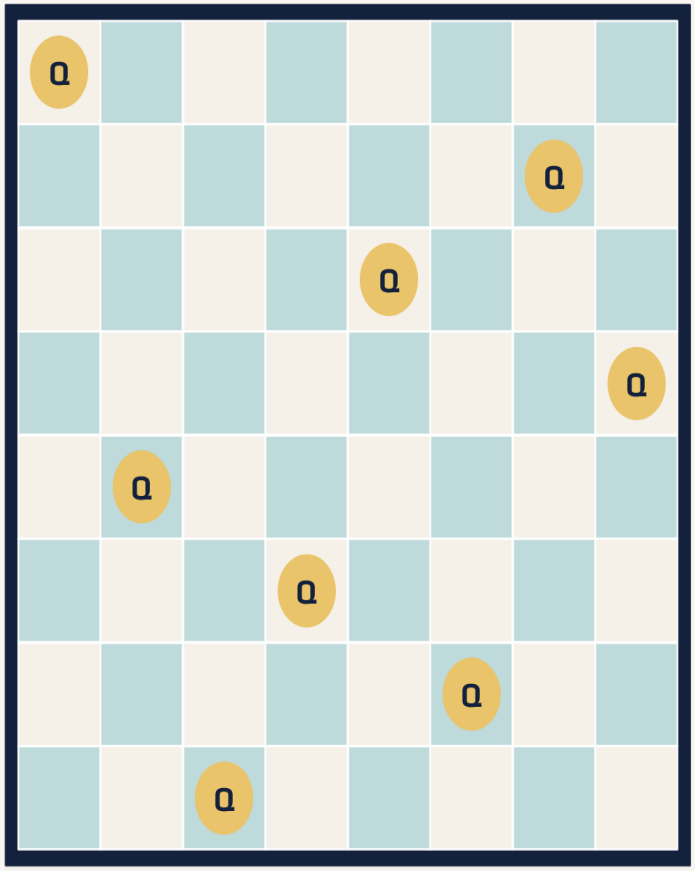
Constraints

- no shared row
- no shared column
- no shared diagonal

Search state

rows_by_column = [0, 4, 7, 5, 2, 6, 1, 3]

N-Queens turns placement into a constraint search



8-QUEENS EXAMPLE

Goal

Place N queens on an N x N board.

Constraints

- no shared row
- no shared column
- no shared diagonal

Search state

rows_by_column = [0, 4, 7, 5, 2, 6, 1, 3]

Structure removes millions of useless placements

One row choice per column

N^N

For N = 8
16,777,216 assignments

Unique rows before diagonals

N!

For N = 8
40,320 permutations

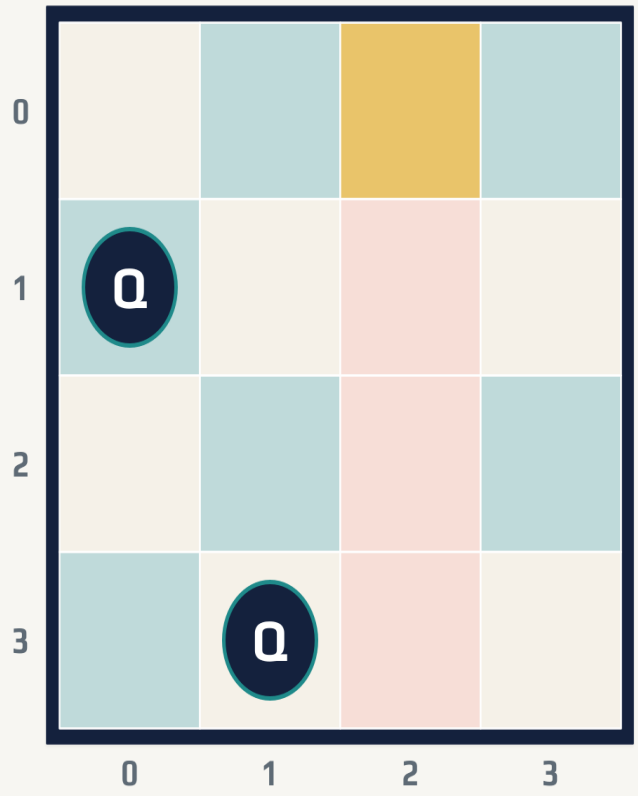
Backtracking

PRUNE

Reject a diagonal conflict as soon as it appears.

Worst-case time remains O(N!), but early checks avoid most complete permutations.

Place one queen per column and reject immediately



COLUMN 2: ROW 0 IS SAFE

Reject a candidate if...

- **row** is already used
- **r - c** diagonal is already used
- **r + c** diagonal is already used

Current conflict sets

rows = {1, 3}
r - c = {1, 2}
r + c = {1, 4}

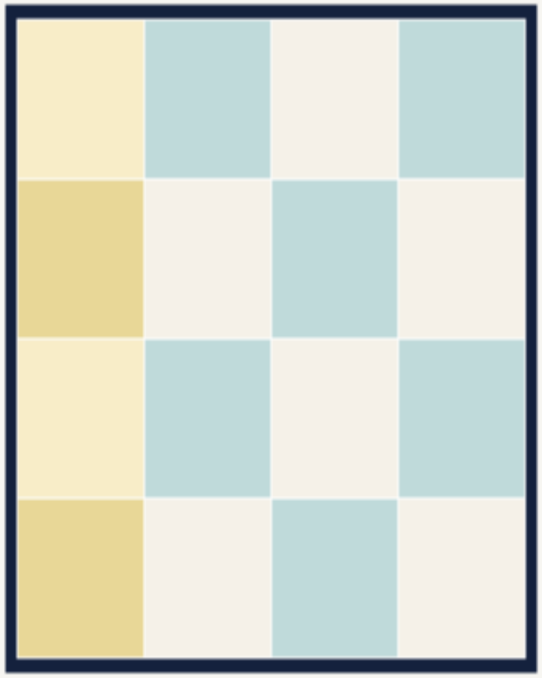
A failed branch undoes, then tries the next row

ANIMATED 4-QUEENS TRACE

Predict first

Why does the branch [0, 2] stop at column 2?

- Every row is used or attacked.
- Remove the most recent queen.
- If no option remains, backtrack again.



Solution found: [1, 3, 0, 2]

The animation loops automatically in Slide Show mode.

Three sets make every safety check constant-time

```
rows, down, up = set(), set(), set()

def place(col):
    if col == N: return True

    for row in range(N):
        if row in rows or row-col in down or row+col in up:
            continue

        rows.add(row); down.add(row-col); up.add(row+col)
        if place(col + 1): return True
        rows.remove(row); down.remove(row-col); up.remove(row+col)

    return False
```

row

r

down diagonal

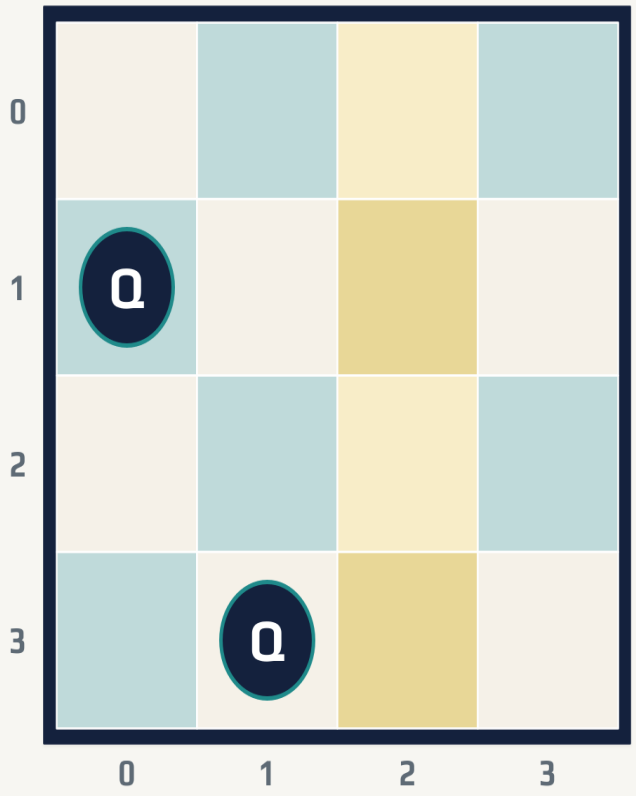
r - c

up diagonal

r + c

Worst case: O(N!) time
Recursion + sets: O(N) space

Your turn: find the only safe row in column 2



NEXT COLUMN = 2

Which row can receive the next queen?

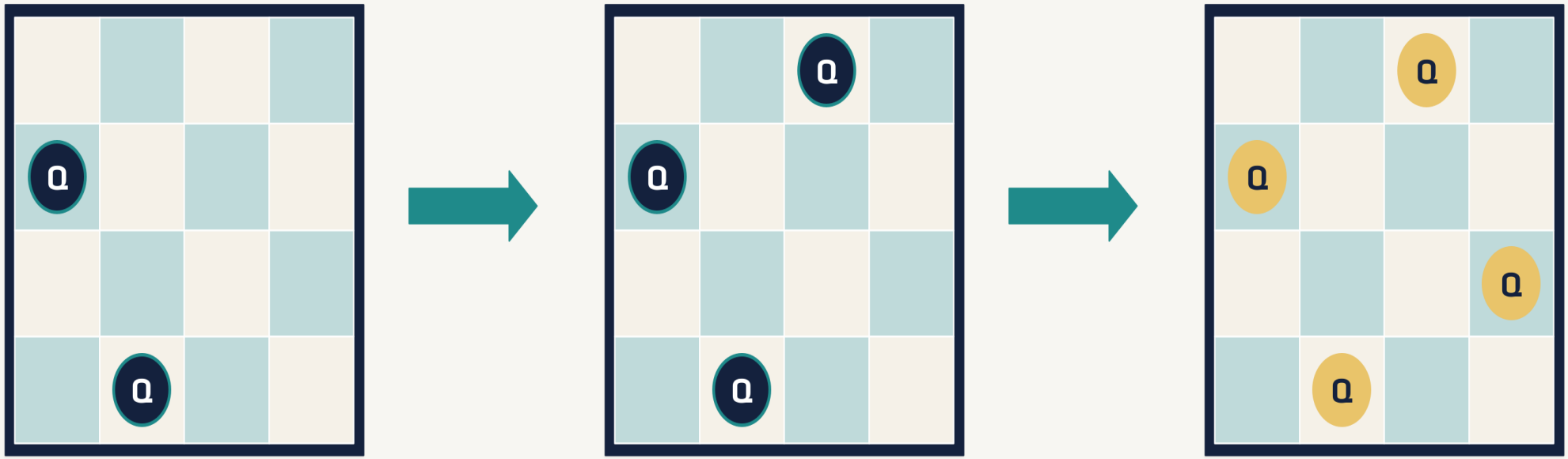
Vote: row 0 row 1 row 2 row 3

Use the current sets

rows = {1, 3}
r - c = {1, 2}
r + c = {1, 4}

Commit before advancing.

Row 0 is safe, then row 2 completes the board



COLUMNS 0-1

ROW 0

ROW 2 / GOAL

place(2) -> row 0 -> place(3) -> row 2 -> GOAL

Row 0 is absent from rows, r-c, and r+c; the recursion can continue.

Full 15-puzzle pseudocode: start with the driver

DRIVER

```
FUNCTION SOLVE_15_PUZZLE(start, goal, max_depth):
    IF KEY(start) = KEY(goal):
        RETURN empty list

    IF NOT IS_SOLVABLE(start):
        RETURN FAILURE

    FOR limit FROM 0 TO max_depth:
        path = empty list
        on_path = { KEY(start) }
        best_depth = { KEY(start): 0 }

        // DLS shares goal, path, on_path and best_depth.
        IF DLS(start, 0, limit):
            RETURN path

    RETURN FAILURE
```

SOLVABILITY GUARD

```
FUNCTION IS_SOLVABLE(board):
    tiles = FLATTEN(board) without BLANK
    inversions = 0

    FOR i FROM 0 TO LENGTH(tiles) - 1:
        FOR j FROM i + 1 TO LENGTH(tiles) - 1:
            IF tiles[i] > tiles[j]:
                inversions = inversions + 1

    N = number of rows
    IF N is odd:
        RETURN inversions is even

    blank_from_bottom = N - ROW_OF_BLANK(board)
    RETURN (inversions + blank_from_bottom) is odd

// Assumes the goal blank is bottom-right.
```

Iterative deepening supplies the limit; parity rejects impossible inputs before recursion.

Full 15-puzzle pseudocode: recurse, prune, and undo

DEPTH-LIMITED SEARCH

```
FUNCTION DLS(state, depth, limit):
    IF KEY(state) = KEY(goal): RETURN TRUE
    IF depth = limit: RETURN FALSE
    FOR EACH move IN LEGAL_MOVES(state):
        next = APPLY_MOVE(state, move)
        key = KEY(next)
        next_depth = depth + 1
        IF key IN on_path: CONTINUE
        IF key IN best_depth AND
            best_depth[key] <= next_depth:
            CONTINUE
        best_depth[key] = next_depth
        ADD key TO on_path
        APPEND move TO path
        IF DLS(next, next_depth, limit):
            RETURN TRUE
        REMOVE LAST ITEM FROM path
        REMOVE key FROM on_path
    RETURN FALSE
```

HELPERS

```
FUNCTION LEGAL_MOVES(state):
    (r, c) = position of BLANK
    moves = empty list
    IF r > 0:     APPEND U
    IF r < N - 1: APPEND D
    IF c > 0:     APPEND L
    IF c < N - 1: APPEND R
    RETURN moves

FUNCTION APPLY_MOVE(state,
move):
    next = COPY(state)
    SWAP BLANK with its neighbor
        in direction move
    RETURN next

FUNCTION KEY(state):
    RETURN immutable flattened
tuple
        of all board cells
```

The undo step removes only the current child from path and on_path.

Full N-Queens pseudocode: initialize the search state

DRIVER

```
FUNCTION SOLVE_N_QUEENS(N):
    board = array of N EMPTY values
    // board[col] stores the selected row

    used_rows = empty set
    used_down = empty set    // row - col
    used_up = empty set      // row + col

    // PLACE and IS_SAFE share the state above.
    IF PLACE(0):
        RETURN board

    RETURN FAILURE
```

CONSTANT-TIME SAFETY TEST

```
FUNCTION IS_SAFE(row, col):
    IF row IN used_rows:
        RETURN FALSE
    IF (row - col) IN used_down:
        RETURN FALSE
    IF (row + col) IN used_up:
        RETURN FALSE
    RETURN TRUE

INVARIANT:
    Columns 0 through col - 1 each contain
    exactly one queen, and no two attack.
```

Rows and the two diagonal keys are the only conflict information the recursion needs.

Full N-Queens pseudocode: place, recurse, and remove

RECURSIVE CORE

```
FUNCTION PLACE(col):
    IF col = N: RETURN TRUE
    FOR row FROM 0 TO N - 1:
        IF NOT IS_SAFE(row, col): CONTINUE
        board[col] = row
        ADD row       TO used_rows
        ADD row - col TO used_down
        ADD row + col TO used_up
        IF PLACE(col + 1):
            RETURN TRUE
        REMOVE row       FROM used_rows
        REMOVE row - col FROM used_down
        REMOVE row + col FROM used_up
        board[col] = EMPTY
    RETURN FALSE
```

A successful call keeps the queens; a failed call removes every change before trying the next row.

Same recursion, different rejection rules

15-PUZZLE

- **State:** full tile arrangement
- **Choice:** move the blank
- **Reject:** illegal move, cycle, depth
- **Undo:** reverse swap and pop path
- **Goal:** target arrangement

N-QUEENS

- **State:** rows chosen by column
- **Choice:** place in the next column
- **Reject:** used row or diagonal
- **Undo:** remove queen and set keys
- **Goal:** N queens placed

CHOOSE -> CHECK -> EXPLORE -> UNDO

Backtracking is the shared pattern. The pruning rule comes from the problem.